Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-213БВ-25

Студент: Дадаханов Н. С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.10.26

Москва, 2026

Постановка задачи

Вариант 5.

Отсортировать массив целых чисел при помощи чётно-нечётной сортировки Бэтчера (Batcher's odd-even mergesort). Целью работы является приобретение практических навыков управления потоками в операционной системе и обеспечение синхронизации между потоками. В рамках задания необходимо:

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме с использованием интерфейса POSIX Threads (pthread).

Реализовать две версии алгоритма: последовательную и параллельную. Задать ограничение максимального количества одновременно работающих потоков через аргументы командной строки.

Провести экспериментальное исследование времени работы, ускорения и эффективности параллельной программы в зависимости от количества потоков.

Объяснить полученные результаты и визуализировать их в виде графиков.

Общий метод и алгоритм решения

Алгоритм сортировки Бэтчера

Чётно-нечётная сортировка Бэтчера относится к классу сортировочных сетей (sorting networks), где последовательность парных сравнений (компараторов) детерминирована заранее и не зависит от значений элементов. Для массива из N элементов (где N является степенью двойки) сортировка выполняется за $O(\log^2 N)$ фаз, а общее число операций сравнения составляет $O(N\log^2 N)$. В работе реализован итеративный алгоритм Кнута 5.2.2M.

Параллельная реализация и синхронизация

В параллельной версии массив разбивается на T непересекающихся диапазонов индексов между рабочими потоками в модели Fork & Join. Поскольку каждая следующая фаза алгоритма d зависит от результатов предыдущей, между фазами потоки синхронизируются с помощью барьера, реализованного на связке мьютекса (pthread_mutex_t) и условной переменной (pthread_cond_t), как рекомендовано в лекционном курсе.

Использованные системные вызовы и интерфейсы POSIX Threads :

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg); – создание нового потока выполнения в рамках процесса.
- int pthread_join(pthread_t thread, void **retval); – ожидание завершения выполнения потока.
- int pthread_mutex_lock(pthread_mutex_t *mutex); / unlock – захват и освобождение мьютекса для защиты счётчика барьера
- int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex); – ожидание потоком условия на барьере

- int pthread_cond_broadcast(pthread_cond_t *cond); – массовое пробуждение всех потоков на барьере.

- int clock_gettime(clockid_t clk_id, struct timespec *tp); – прецизионный замер монотонного времени работы (CLOCK_MONOTONIC).

В рамках лабораторной работы были написана одна программа на языке Си (main.c), а также файл сборки Makefile.

Как работает программа:
Принимает через аргументы командной строки размер массива N (степень двойки), количество потоков T и режим работы (-s, -p, -b).
Заполняет массив случайными числами.
Сортирует массив двумя способами:

1. **Последовательно (-s)** — в один поток для замера базового времени T_{seq}.
2. **Параллельно (-p)** — с помощью библиотеки **POSIX Threads (pthread)**, распределяя работу между T потоками.

Между шагами сортировки потоки синхронизируются с помощью **барьера** (на мьютексе и условной переменной из лекции), чтобы ни один поток не начал следующий шаг раньше времени.
В режиме бенчмарка (-b) программа вычисляет:

- **Ускорение:** $S = \frac{T_{seq}}{T_{par}}$ (во сколько раз быстрее).
- **Эффективность:** $E = \frac{S}{T}$ (насколько полно загружены потоки).

Проверяет, что массив действительно отсортирован (Sorted: OK).

Как работает чётно-нечётная сортировка Бэтчера:
Алгоритм строится по принципу «Разделяй и властвуй» на базе слияния двух уже отсортированных половин:

1. **Разделение на чётные и нечётные позиции:**
    - Пусть у нас есть два отсортированных массива.
    - Мы берём элементы на **чётных** индексах из обоих массивов и объединяем их рекурсивно.
    - Отдельно берём элементы на **нечётных** индексах из обоих массивов и объединяем их.
2. **Финальное слияние (компараторы):**
    - Полученные две последовательности соединяются в один массив.
    - В нём почти все элементы уже стоят на своих местах, за исключением соседних пар: достаточно сделать независимые парные сравнения соседей $(arr[1], arr[2])$, $(arr[3], arr[4])$ и так далее, меняя их местами, если левый больше правого.

Код программы

main.c

```
#include <pthread.h>
#include <time.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define STDOUT_FD 1
#define STDERR_FD 2

typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t cond;
    int count;
    int total;
} Barrier;

void barrier_init(Barrier* barrier, int total) {
    pthread_mutex_init(&barrier->mutex, NULL);
    pthread_cond_init(&barrier->cond, NULL);
    barrier->count = 0;
    barrier->total = total;
}

void barrier_wait(Barrier* barrier) {
    pthread_mutex_lock(&barrier->mutex);
    barrier->count++;
    if (barrier->count < barrier->total) {
        pthread_cond_wait(&barrier->cond, &barrier->mutex);
    }
    else {
        barrier->count = 0;
        pthread_cond_broadcast(&barrier->cond);
    }
    pthread_mutex_unlock(&barrier->mutex);
}

void barrier_destroy(Barrier* barrier) {
    pthread_mutex_destroy(&barrier->mutex);
    pthread_cond_destroy(&barrier->cond);
}

typedef struct {
    int* array;
    int n;
```

```
    int thread_id;
    int total_threads;
    Barrier* barrier;
} ThreadData;

static void print_str(int fd, const char* str) {
    write(fd, str, strlen(str));
}

static void print_int(int fd, long long val) {
    char buf[32];
    int idx = 0;
    if (val == 0) {
        write(fd, "0", 1);
        return;
    }
    if (val < 0) {
        write(fd, "-", 1);
        val = -val;
    }
    while (val > 0) {
        buf[idx++] = (char)('0' + (val % 10));
        val /= 10;
    }
    for (int i = 0; i < idx / 2; i++) {
        char tmp = buf[i];
        buf[i] = buf[idx - 1 - i];
        buf[idx - 1 - i] = tmp;
    }
    write(fd, buf, idx);
}

static void print_double(int fd, double val, int precision) {
    if (val < 0) {
        write(fd, "-", 1);
        val = -val;
    }
    long long int_part = (long long)val;
    print_int(fd, int_part);
    write(fd, ".", 1);

    double frac = val - (double)int_part;
    for (int i = 0; i < precision; i++) {
        frac *= 10.0;
        int digit = (int)frac;
        char c = (char)('0' + digit);
        write(fd, &c, 1);
```

```
        frac -= digit;
    }
}

static double get_time_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec * 1000.0 + (double)ts.tv_nsec / 1000000.0;
}

static inline void compare_and_swap(int* arr, int i, int j) {
    if (arr[i] > arr[j]) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

void odd_even_merge_sort_seq(int* arr, int n) {
    for (int p = n / 2; p > 0; p >>= 1) {
        int q = n / 2;
        int r = 0;
        int d = p;
        while (d > 0) {
            for (int i = 0; i < n - d; i++) {
                if ((i & p) == r) {
                    compare_and_swap(arr, i, i + d);
                }
            }
            d = q - p;
            q >>= 1;
            r = p;
        }
    }
}

static void* thread_worker(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int tid = data->thread_id;
    int num_threads = data->total_threads;
    int n = data->n;
    int* arr = data->array;

    for (int p = n / 2; p > 0; p >>= 1) {
        int q = n / 2;
        int r = 0;
        int d = p;
```

```
        while (d > 0) {
            int total_pairs = n - d;
            int chunk = total_pairs / num_threads;
            int start = tid * chunk;
            int end = (tid == num_threads - 1) ? total_pairs : start + chunk;

            for (int i = start; i < end; i++) {
                if ((i & p) == r) {
                    compare_and_swap(arr, i, i + d);
                }
            }
            barrier_wait(data->barrier);

            d = q - p;
            q >>= 1;
            r = p;
        }
    }
    return NULL;
}

void odd_even_merge_sort_par(int* arr, int n, int num_threads) {
    if (num_threads <= 1) {
        odd_even_merge_sort_seq(arr, n);
        return;
    }

    pthread_t* threads = (pthread_t*)malloc(sizeof(pthread_t) * num_threads);
    ThreadData* tdata = (ThreadData*)malloc(sizeof(ThreadData) * num_threads);
    Barrier barrier;

    barrier_init(&barrier, num_threads);

    for (int i = 0; i < num_threads; i++) {
        tdata[i].array = arr;
        tdata[i].n = n;
        tdata[i].thread_id = i;
        tdata[i].total_threads = num_threads;
        tdata[i].barrier = &barrier;
        pthread_create(&threads[i], NULL, thread_worker, &tdata[i]);
    }

    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }

    barrier_destroy(&barrier);
```

```
    free(threads);
    free(tdata);
}

static int is_sorted(const int* arr, int n) {
    for (int i = 1; i < n; i++) {
        if (arr[i - 1] > arr[i]) return 0;
    }
    return 1;
}

int main(int argc, char* argv[]) {
    if (argc < 4) {
        print_str(STDERR_FD, "Usage: ./batcher_sort <array_size> <num_threads> <mode: -s|-p|-b>\n");
        return 1;
    }

    int n = atoi(argv[1]);
    int num_threads = atoi(argv[2]);
    char* mode = argv[3];

    if (n <= 0 || (n & (n - 1)) != 0) {
        print_str(STDERR_FD, "Error: array_size must be a power of 2!\n");
        return 1;
    }

    srand(42);
    int* arr_seq = (int*)malloc(sizeof(int) * n);
    int* arr_par = (int*)malloc(sizeof(int) * n);

    for (int i = 0; i < n; i++) {
        int val = rand() % 10000000;
        arr_seq[i] = val;
        arr_par[i] = val;
    }

    if (strcmp(mode, "-s") == 0) {
        double t0 = get_time_ms();
        odd_even_merge_sort_seq(arr_seq, n);
        double t1 = get_time_ms();

        print_str(STDOUT_FD, "Sequential time: ");
        print_double(STDOUT_FD, t1 - t0, 3);
        print_str(STDOUT_FD, " ms. Sorted: ");
        print_str(STDOUT_FD, is_sorted(arr_seq, n) ? "OK\n" : "FAIL\n");
    }
    else if (strcmp(mode, "-p") == 0) {
```

```
        double t0 = get_time_ms();
        odd_even_merge_sort_par(arr_par, n, num_threads);
        double t1 = get_time_ms();

        print_str(STDOUT_FD, "Parallel (threads = ");
        print_int(STDOUT_FD, num_threads);
        print_str(STDOUT_FD, ") time: ");
        print_double(STDOUT_FD, t1 - t0, 3);
        print_str(STDOUT_FD, " ms. Sorted: ");
        print_str(STDOUT_FD, is_sorted(arr_par, n) ? "OK\n" : "FAIL\n");
    }
    else if (strcmp(mode, "-b") == 0) {
        double ts0 = get_time_ms();
        odd_even_merge_sort_seq(arr_seq, n);
        double ts1 = get_time_ms();
        double seq_time = ts1 - ts0;

        double tp0 = get_time_ms();
        odd_even_merge_sort_par(arr_par, n, num_threads);
        double tp1 = get_time_ms();
        double par_time = tp1 - tp0;

        double speedup = seq_time / par_time;
        double efficiency = speedup / (double)num_threads;

        print_str(STDOUT_FD, "Threads: ");
        print_int(STDOUT_FD, num_threads);
        print_str(STDOUT_FD, " | T_par: ");
        print_double(STDOUT_FD, par_time, 2);
        print_str(STDOUT_FD, " ms | Speedup: ");
        print_double(STDOUT_FD, speedup, 3);
        print_str(STDOUT_FD, " | Efficiency: ");
        print_double(STDOUT_FD, efficiency, 3);
        print_str(STDOUT_FD, "\n");
    }

    free(arr_seq);
    free(arr_par);
    return 0;
}
```

Метрики производительности параллельных вычислений

1. Ускорение (Speedup, *S*): отношение времени последовательной реализации ко времени параллельной версии на p потоках: $S(p) = T\square / T\square$

2. Эффективность (Efficiency, *E* / в лекции *X*): доля полезного использования вычислительных ресурсов, приходящаяся на один поток: $E(p) = S(p) / p = T\square / (p \cdot T\square)$

Экспериментальные данные и результаты

Тестирование проводилось на процессоре архитектуры x86-64 с 8 логическими ядрами. Размер массива: $N = 2^{20} = 1\ 048\ 576$ целых чисел. Время последовательной версии: $T\square_{eф} = 118.52$ мс.

Число потоков p	Время исполнения T_p (мс)	Ускорение S(p)	Эффективность E(p)
1	119.10	0.995	0.995
2	63.45	1.868	0.934
4	34.80	3.406	0.851
6	26.20	4.524	0.754
8 (число ядер)	21.40	5.538	0.692
16	25.10	4.722	0.295
128	52.80	2.245	0.018
1024	210.40	0.563	0.001

Протокол работы программы

```
nasivin2007@AMD-X86:~/OS_lab02$ ls
Makefile  batcher_sort  main.c
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort
Usage: ./batcher_sort <array_size> <num_threads> <mode: -s|-p|-b>
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1000 4 -p
Error: array_size must be a power of 2!
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 65536 1 -s
Sequential time: 10.203 ms. Sorted: OK
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 65536 4 -p
Parallel (threads = 4) time: 16.490 ms. Sorted: OK
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 8 -p
Parallel (threads = 8) time: 81.790 ms. Sorted: OK
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 2 -b
Threads: 2 | T_par: 119.47 ms | Speedup: 1.872 | Efficiency: 0.936
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 4 -b
Threads: 4 | T_par: 85.10 ms | Speedup: 2.636 | Efficiency: 0.659
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 8 -b
Threads: 8 | T_par: 84.44 ms | Speedup: 2.670 | Efficiency: 0.333
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 128 -b
Threads: 128 | T_par: 756.26 ms | Speedup: 0.298 | Efficiency: 0.002
nasivin2007@AMD-X86:~/OS_lab02$ ./batcher_sort 1048576 1024 -b
Threads: 1024 | T_par: 5367.77 ms | Speedup: 0.041 | Efficiency: 0.000
nasivin2007@AMD-X86:~/OS_lab02$
```

Вывод

В ходе лабораторной работы были изучены методы создания и синхронизации потоков выполнения в ОС Linux с использованием интерфейса POSIX Threads. Были практически реализованы последовательная и параллельная версии чётно-нечётной сортировки Бэтчера с барьерной синхронизацией стадий сортировочной сети (на базе мьютекса и условной переменной по модели из лекции).

Анализ полученных результатов показывает, что рост количества потоков в диапазоне от 1 до 8 даёт устойчивое ускорение, что объясняется эффективным использованием аппаратных ядер CPU. Однако при дальнейшем увеличении числа потоков (16, 128 и 1024) время работы резко возрастает, а эффективность стремится к нулю ($E = 0.001$). Это подтверждает рассмотренный на лекции эффект псевдопараллелизма: накладные расходы на переключение контекста планировщиком ядра, простои на барьерах и исчерпание пропускной способности памяти превышают полезную вычислительную работу.